Programación de SE en C

(Parte II: Máquinas de estados finitos)

Máquinas de estados finitos (FSM)

- Permiten modelar un problema particular, **con mayor nivel de abstracción**
- Muy útiles para representar sistemas **reactivos a eventos** o con **muchos modos de funcionamiento**
- Pueden **simularse** y evaluarse antes de implementarse en C
- Existen distintas maneras de implementar las FSM en código:

  * switch/if anidados

  * funciones de estados

  * Tablas de transiciones

Máquinas de estados finitos (FSM)

- Son compatibles con las distintas arquitecturas de software:

  * super-loop - foreground/background

  *** event-driven**

  * time triggered

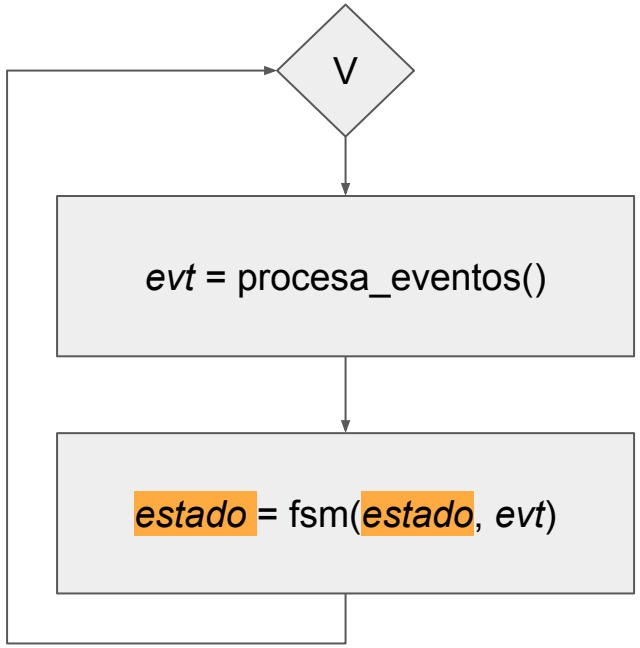
  * RTOS

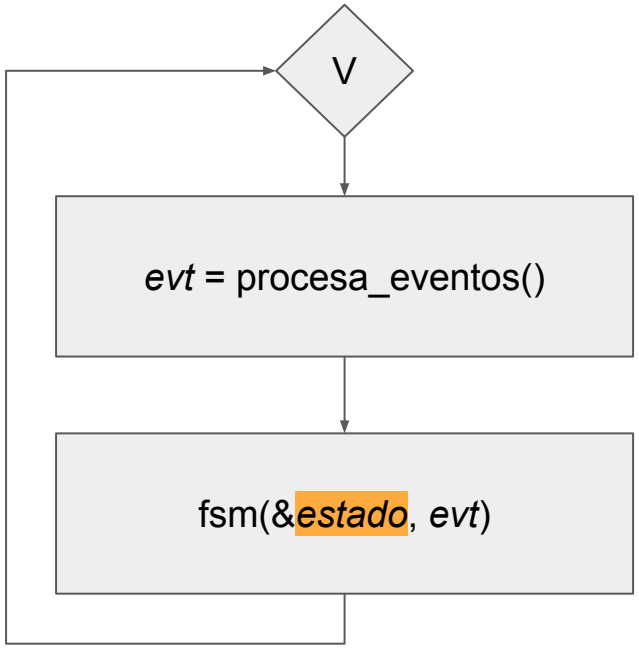
- **Se debe procesar POR COMPLETO de a un evento a la vez**. Por lo que se debe tener especial cuidado con las tareas concurrentes e interrupciones.

- Suelen utilizarse colas/buffers para el procesamiento de eventos

Máquinas de estados finitos (FSM)

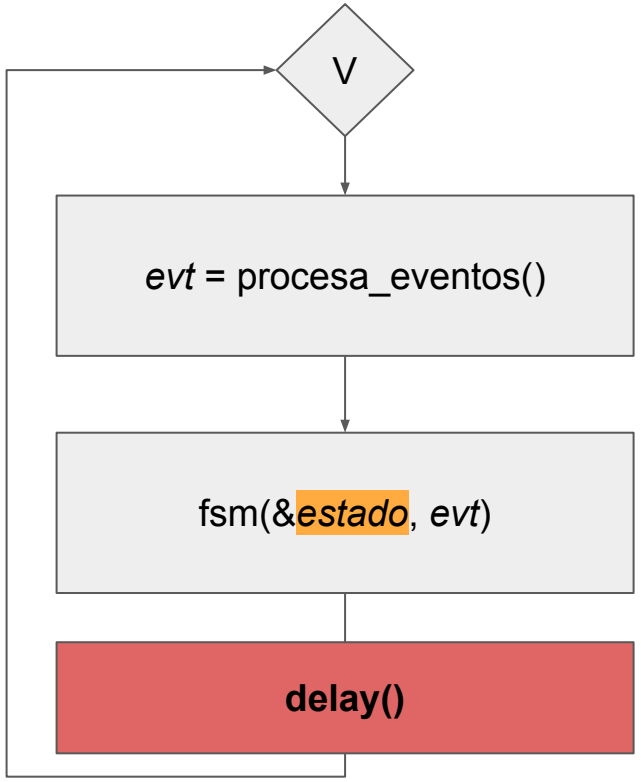
- super-loop





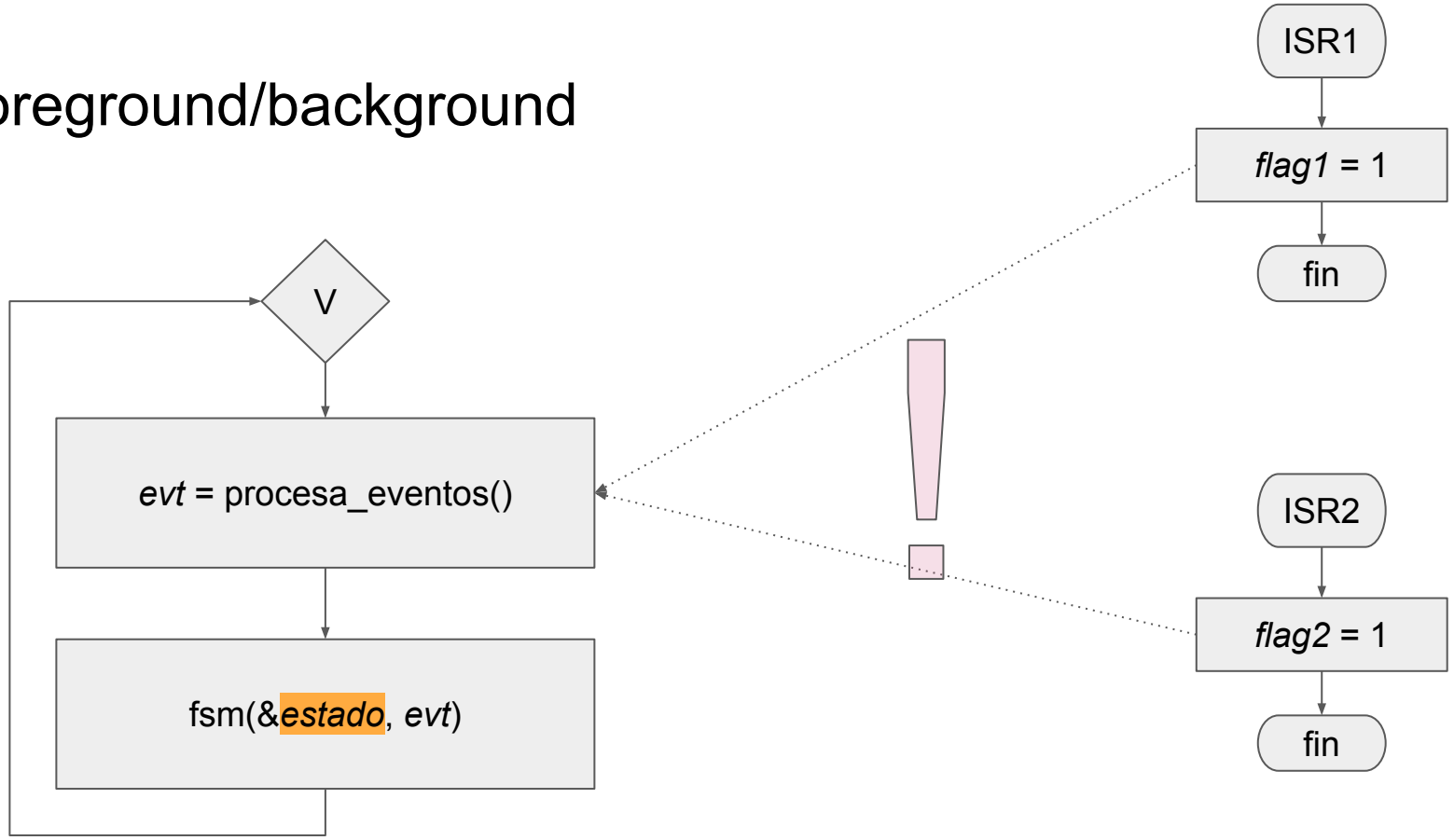
Máquinas de estados finitos (FSM)

- super-loop



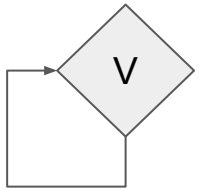
Máquinas de estados finitos (FSM)

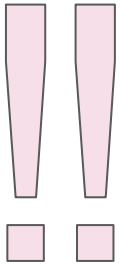
- foreground/background

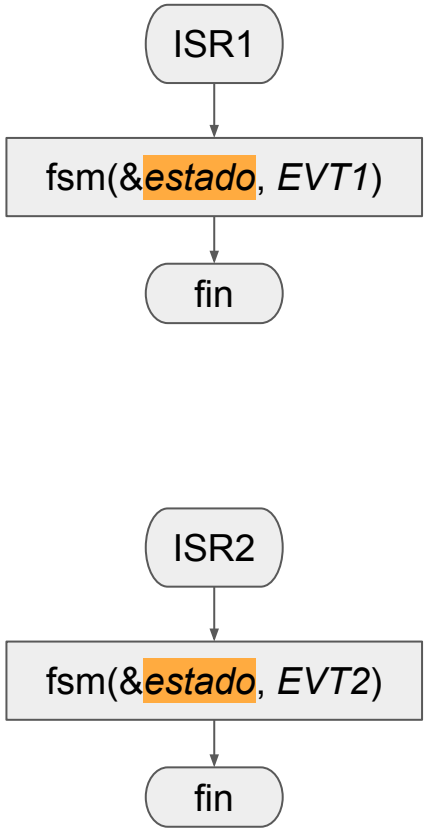


Máquinas de estados finitos (FSM)

- foreground/background

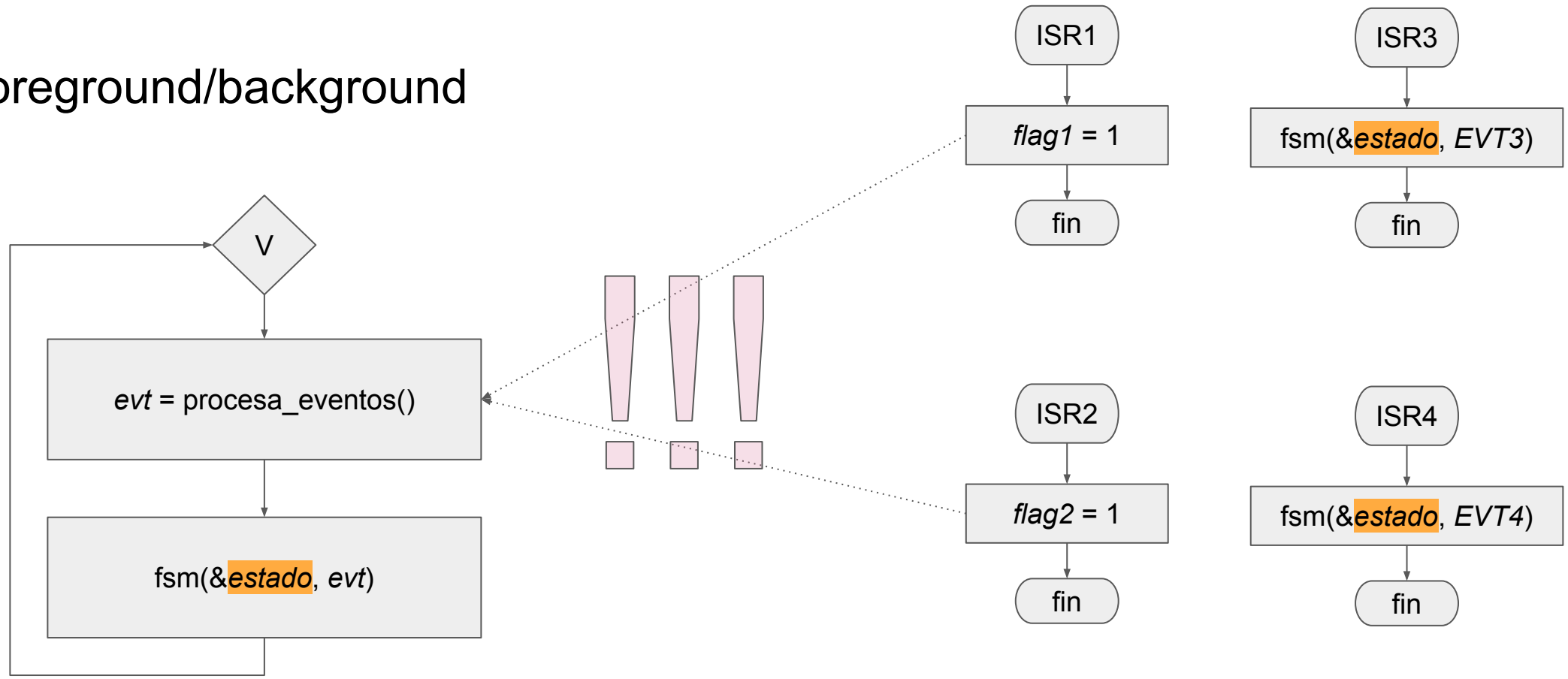






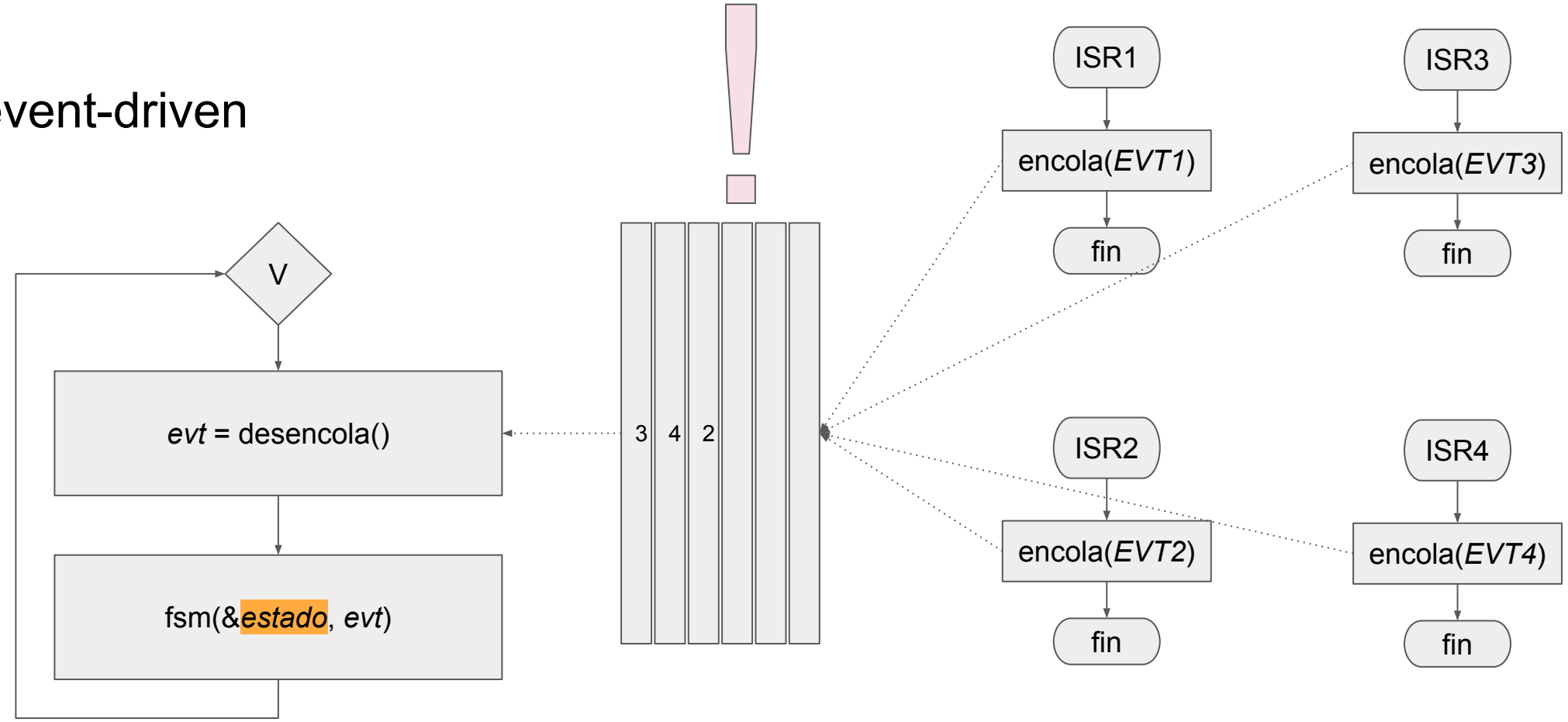
Máquinas de estados finitos (FSM)

- foreground/background



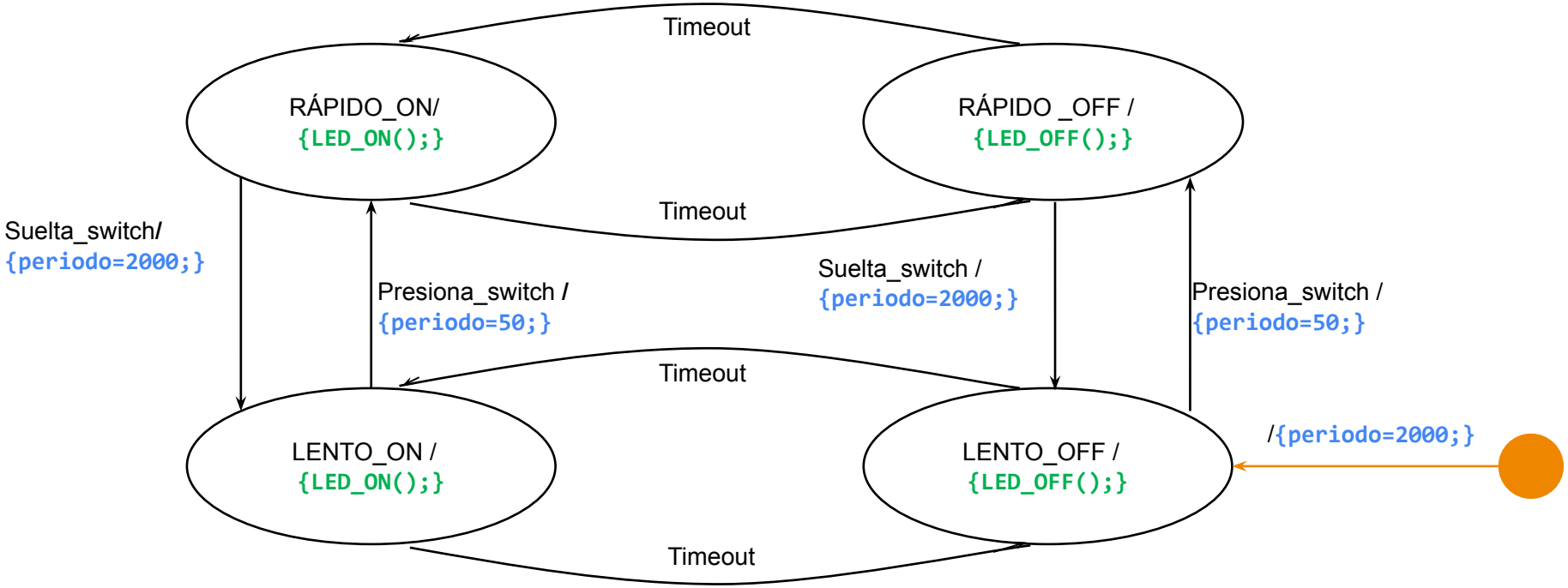
Máquinas de estados finitos (FSM)

- event-driven



Máquinas de estados finitos (FSM)

- Las FSM <u>siempre deben estar en un estado definido</u>, **desde el inicio**
- Los estados suelen ser puntos de reposo donde no se modifican las salidas (no se ejecutan acciones)
- Las acciones (salidas) pueden estar asociadas a las **transiciones** (Mealy), al **ingreso al próximo estado** (Moore)



Máquinas de estados finitos (FSM)
Implementación con switch anidados

- Conviene crear un **tipo enum** para los estados y otro para los eventos*

```
typedef enum  {RAPIDO_ON, RAPIDO_OFF, LENTO_ON, LENTO_OFF}estados_e;
typedef enum {PRESIONA_SW, SUELTA_SW, TIMEOUT, NULL_EVT}eventos_e;*
```

- Y una **variable** que indicará el **estado actual** de la FSM (inicializada según el estado inicial)

```
estados_e estado_actual = LENTO_OFF;
```

- El código de la FSM puede modularizarse en una función que recibe el estado actual como parámetro por referencia y el evento que se desea procesar*

```
void fsm_switch(estados_e *estado, eventos_e evento);
```

(también puede usar el valor de retorno para devolver el nuevo estado)

```
estados_e fsm_switch(estados_e estado, eventos_e evento);
```

- Cada vez que se invoca, se evalúa **solo para el estado actual** el evento* pasado como parámetro y solo si corresponde realiza una transición hacia otro estado

Máquinas de est
Implementación

*** hay máquinas de estados que hacen polling de entradas y no están manejadas por eventos (evalúan condiciones). Otras sólo tienen un único evento.**

- Conviene crear

```
typedef enum  {RAPIDO_ON, RAPIDO_OFF, LENTO_ON, LENTO_OFF}estados_e;
typedef enum {PRESIONA_SW, SUELTA_SW, TIMEOUT, NULL_EVT}eventos_e;*
```

- Y una **variable** que indicará el **estado actual** de la FSM (inicializada según el estado inicial)

```
estados_e estado_actual = LENTO_OFF;
```

- El código de la FSM puede modularizarse en una función que recibe el estado actual como parámetro por referencia y el evento que se desea procesar*

```
void fsm_switch(estados_e *estado, eventos_e evento);
```

(también puede usar el valor de retorno para devolver el nuevo estado)

```
estados_e fsm_switch(estados_e estado, eventos_e evento);
```

- Cada vez que se invoca, se evalúa **solo para el estado actual** el evento* pasado como parámetro y solo si corresponde realiza una transición hacia otro estado

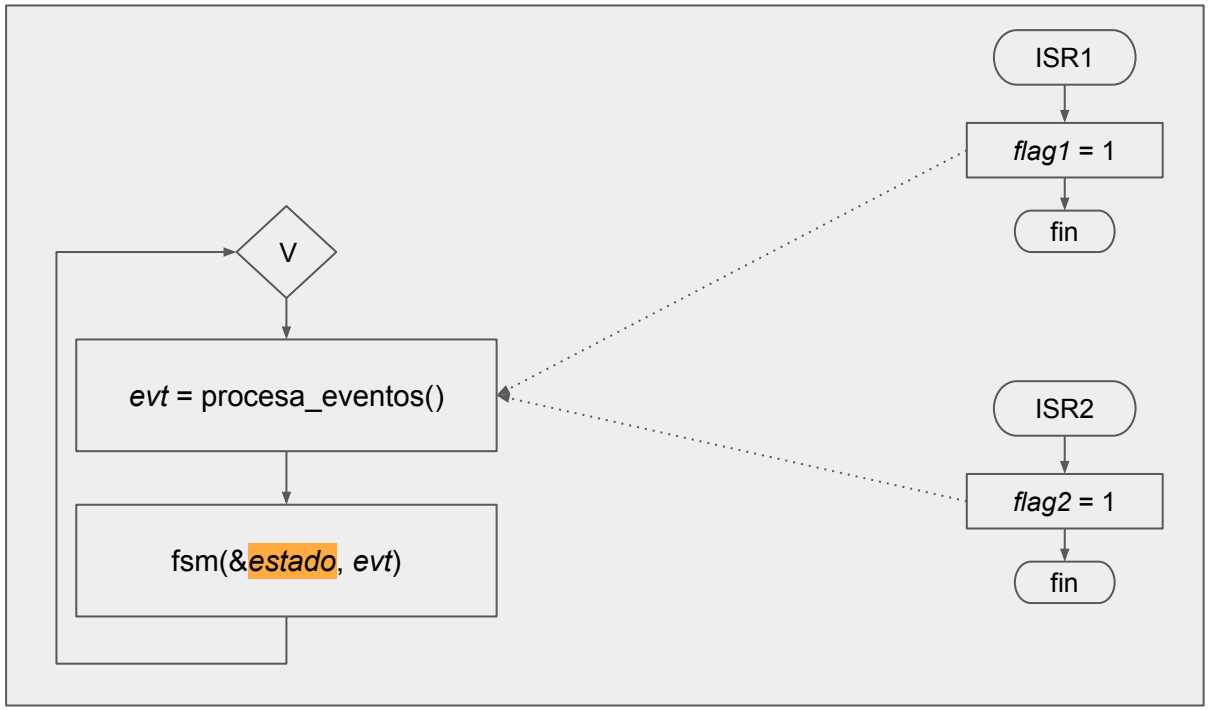
```c
void fsm_switch(estados_e *actual, eventos_e evento){
  estados_e estado_anterior = *actual;
  //DETECTO TRANSICIONES  Y EJECUTO ACCIONES EN LA TRANSICIÓN
  //TIPO MÁQUINA DE MEALY
  switch (*actual) {
    case LENTO_OFF:
      switch(evento){
        case PRESIONA_SW:
          htim3.Instance->ARR = 50;
          htim3.Instance->EGR |= 1;
          *actual = RAPIDO_OFF;
          break;
        case TIMEOUT:
          *actual = LENTO_ON;
          break;
      }
      break;
    case LENTO_ON:
      switch(evento){
        case PRESIONA_SW:
          //...
        case TIMEOUT:
          //...
      }
      break;
    case RAPIDO_OFF:
      //...
    case RAPIDO_ON:
      //...
  }
```

```c
  //SI HUBO UN CAMBIO DE ESTADO EJECUTO ACCIONES ASOCIADAS A LA ENTRADA AL ESTADO
  //(TIPO MOORE)
  if (*actual != estado_anterior) {
    switch (*actual) {
      case LENTO_OFF:
      case RAPIDO_OFF:
        HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_SET);
        break;
      case LENTO_ON:
      case RAPIDO_ON:
        HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_RESET);
        break;
    }
  }
}
```

```c
void fsm_switch(estados_e *actual, eventos_e evento){
  estados_e estado_anterior = *actual;
  //DETECTO TRANSICIONES  Y EJECUTO ACCIONES EN LA TRANSICIÓN
  //TIPO MÁQUINA DE MEALY
  switch (*actual) {
    case LENTO_OFF:
      switch(evento){
        case PRESIONA_SW:
          htim3.Instance->ARR = 50;
          htim3.Instance->EGR |= 1;
          *actual = RAPIDO_OFF;
          HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_SET);
          break;
        case TIMEOUT:
          *actual = LENTO_ON;
          HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_RESET);
          break;
      }
      break;
    case LENTO_ON:
      switch(evento){
        case PRESIONA_SW:
          //...
        case TIMEOUT:
          //...
      }
      break;
    case RAPIDO_OFF:
      //...
    case RAPIDO_ON:
      //...
  }
```

Invocación de la FSM - Backgorund

```
estados_e estado_actual = LENTO_OFF;
int main(void) {
  /* ... */
  while (1) {
    //procesa eventos
    eventos_e evt=NULL_EVT;
    if(timeout>0){
      timeout=0;
      evt = TIMEOUT;
    }
    else{ pin = HAL_GPIO_ReadPin(SWITCH_GPIO_Port, SWITCH_Pin);
      if(pin != pin_last)
        evt = (pin==GPIO_PIN_SET)?PRESIONA_SW:SUELTA_SW;
      pin_last = pin;
    }

    //corre la máquina de estados
    fsm_switch(&estado_actual, evt);
  }
}
```



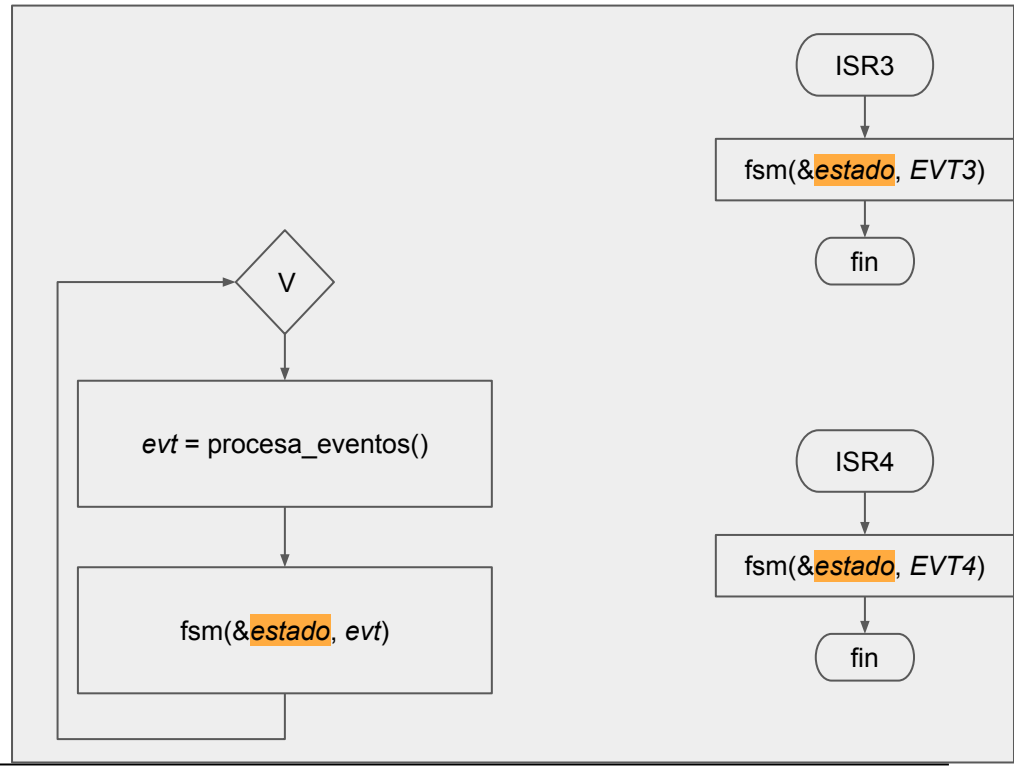

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
  timeout=1;
}
```

Invocación de la FSM - Backgorund + Foreground

Con cuidado de no interrumpir transiciones u acciones en curso!!!

```
estados_e estado_actual = LENTO_OFF;
int main(void) {
  /* ... */
  while (1) {
    pin = HAL_GPIO_ReadPin(SWITCH_GPIO_Port, SWITCH_Pin);
    if(pin != pin_last){
      if(pin==GPIO_PIN_SET)
        __disable_irq();fsm_switch(&estado_actual, PRESIONA_SW);__enable_irq();
      else
        __disable_irq();fsm_switch(&estado_actual, SUELTA_SW);__enable_irq();
    }
    pin_last = pin;
  }
}
```

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
  fsm_switch(&estado_actual, TIMEOUT);
}
```



Implementación de FSM con funciones de estados (usando ***punteros a función***)

Máquinas de estados finitos (FSM) – Implementación con funciones de estados (usando **punteros a función**)

- La FSM puede implementarse “rompiendo” el switch externo de la solución anterior en una función por estado del tipo:

```
void estado(estado_t *actual, eventos_e evento)
```

```
void lento_off(estado_t *actual, eventos_e evento);
void lento_on(estado_t *actual, eventos_e evento);
void rapido_off(estado_t *actual, eventos_e evento);
void rapido_on(estado_t *actual, eventos_e evento);
```

```
o ... estado_t estado(estado_t *actual, eventos_e evento)
```

```
estado_t lento_off(eventos_e evento);
estado_t lento_on(eventos_e evento);
estado_t rapido_off(eventos_e evento);
estado_t rapido_on(eventos_e evento);
```

Máquinas de estados finitos (FSM) – Implementación con funciones de estados (usando **punteros a función**)

- **`estado_t`** es el tipo de dato del puntero a las funciones estado, definido como:

```
typedef void(*estado_t)(estado_t*, eventos_e);
```

o...

```
typedef estado_t(*estado_t)(eventos_e);
```

- Se `define` una variable que indicará el estado actual de la FSM (inicializada según el estado inicial)

```
estado_t fsm = lento_off;
```

Máquinas de estados finitos (FSM) – Implementación con funciones de estados (usando **punteros a función**)

- **`estado_t`** es el tipo de dato del puntero a las funciones estado, definido como:

```
typedef void(*estado_t)(~~estado_t~~*, eventos_e);
                        void*
```

o...

```
typedef ~~estado_t~~(*estado_t)(eventos_e);
        void*
```

en la definición del tipo uso void* como comodín de puntero

- Se `define` una variable que indicará el estado actual de la FSM (inicializada según el estado inicial)

```
estado_t fsm = lento_off;
```

Máquinas de estados finitos (FSM) – Implementación con funciones de estados (usando **punteros a función**)

- La “variable” fsm a la vez que almacena el estado actual, es un puntero a la función que debe ejecutarse de la máquina de estados cuando sucede un evento:

```
fsm(&fsm,evt);                    fsm = fsm(evt);
```

- La ventaja sobre la implementación con switch es una *mayor velocidad de ejecución* y *menor complejidad de implementación* para **máquinas de estados grandes** (con muchos estados)

Máquinas de estados finitos (FSM) – Implementación con funciones de estados (usando **punteros a función**)

```
void lento_off(estado_t *actual, eventos_e evento)
{
  switch(evento){
  case PRESIONA_SW:
    htim3.Instance->ARR = 50;
    htim3.Instance->EGR |= 1;
    *actual = rapido_off;
    HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_SET);
    break;
  case TIMEOUT:
    *actual = lento_on;
    HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_RESET);
    break;
  }
}
```

```
estado_t lento_off(eventos_e evento)
{
  estado_t actual = lento_off;
  switch(evento){
  case PRESIONA_SW:
    htim3.Instance->ARR = 50;
    htim3.Instance->EGR |= 1;
    actual = rapido_off;
    HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_SET);
    break;
  case TIMEOUT:
    actual = lento_on;
    HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_RESET);
    break;
  }
  return actual;
}
```

Implementación de FSM con **tabla de transiciones** (usando ***punteros a función***)

Máquinas de estados finitos (FSM) – Implementación con **tabla de transiciones** (punteros a función)

- Vimos que para un gran número de estados es más eficiente utilizar una función por estado y punteros a función.
- Pero si el número de eventos/entradas a evaluar también es grande se pierde eficiencia en los switch o if/else que verifican la existencia de eventos
- Se logra una implementación más eficiente con una función por cada combinación de estado-evento posible y construir una tabla de doble entrada (una matriz 2D) que a partir del estado actual y del evento ocurrido nos de acceso a un manejador de evento que puede o no terminar en una transición.
- Es decir, hay que construir una matriz 2D de punteros a función, donde la fila indica el estado y la columna el evento

Máquinas de estados finitos (FSM) – Implementación con **tabla de transiciones** (punteros a función)

```
typedef void* (*estado_t)();
estado_t* rapido_on_ssw();
estado_t* rapido_on_to();
estado_t* rapido_off_ssw();
estado_t* rapido_off_to();
estado_t* lento_on_psw();
estado_t* lento_on_to();
estado_t* lento_off_psw();
estado_t* lento_off_to();
estado_t* null(){};
```

```
estado_t* rapido_on_ssw(){
  htim3.Instance->ARR = 2000;
  htim3.Instance->EGR |= 1;
 HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_SET);
  return (estado_t*)tabla[LENTO_ON];
}

estado_t* rapido_on_to(){
  HAL_GPIO_WritePin(LED_GPIO_Port, LED_Pin, GPIO_PIN_RESET);
  return (estado_t*)tabla[RAPIDO_OFF];
}
```

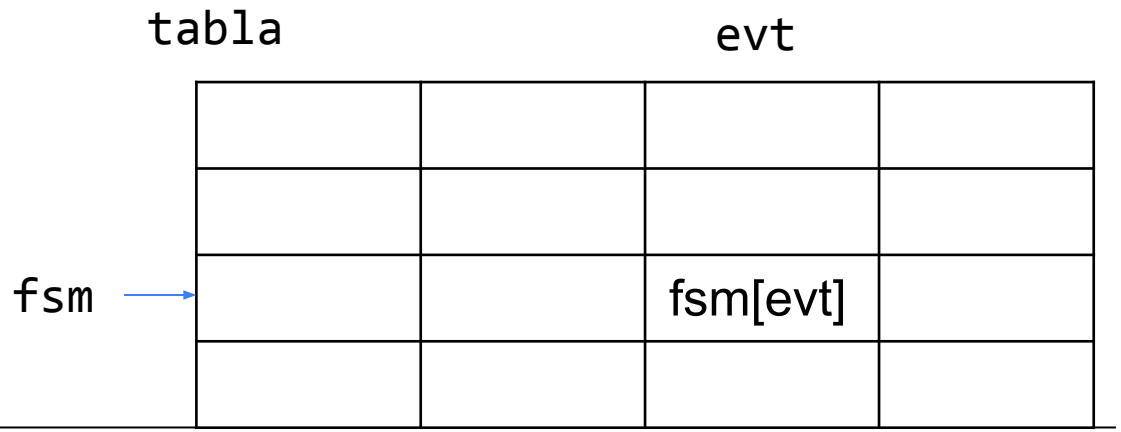
```
typedef enum  {RAPIDO_ON, RAPIDO_OFF, LENTO_ON, LENTO_OFF}estados_e;
typedef enum{NULL_EVT, PRESIONA_SW, SUELTA_SW, TIMEOUT}eventos_e;
#define NUM_EVENTOS 4
#define NUM_ESTADOS 4

estado_t const tabla[NUM_ESTADOS][NUM_EVENTOS] = {
    {null,          null,          rapido_on_ssw,  rapido_on_to},   /* RAPIDO_ON */
    {null,          null,          rapido_off_ssw, rapido_off_to},   /* RAPIDO_OFF */
    {null,          lento_on_psw,  null,           lento_on_to},   /* LENTO_ON */
    {null,          lento_off_psw, null,           lento_off_to}};  /* LENTO_OFF */
                   /* PRESIONA_SW    SUELTA_SW       TIMEOUT   */
```

```
estado_t *fsm = (estado_t)tabla[LENTO_OFF];
```

Máquinas de estados finitos (FSM) – Implementación con **tabla de transiciones** (punteros a función)

```
while (1) {
  eventos_e evt = NULL_EVT;
  if(timeout>0){
    timeout=0;
    evt = TIMEOUT;
   }
  else{
     pin = HAL_GPIO_ReadPin(SWITCH_GPIO_Port, SWITCH_Pin);
     if(pin != pin_last) evt=(pin==GPIO_PIN_SET)?PRESIONA_SW:SUELTA_SW;
  }
  fsm = fsm[evt]();
}
```

- El estado actual queda determinado por la fila a la que apunta fsm.
- Al ocurrir un evento se ejecuta la función apuntada por fsm[evt] y se retorna el nuevo estado como un puntero a otra fila



Statecharts: FSM con extensiones

Extensiones de las FSM

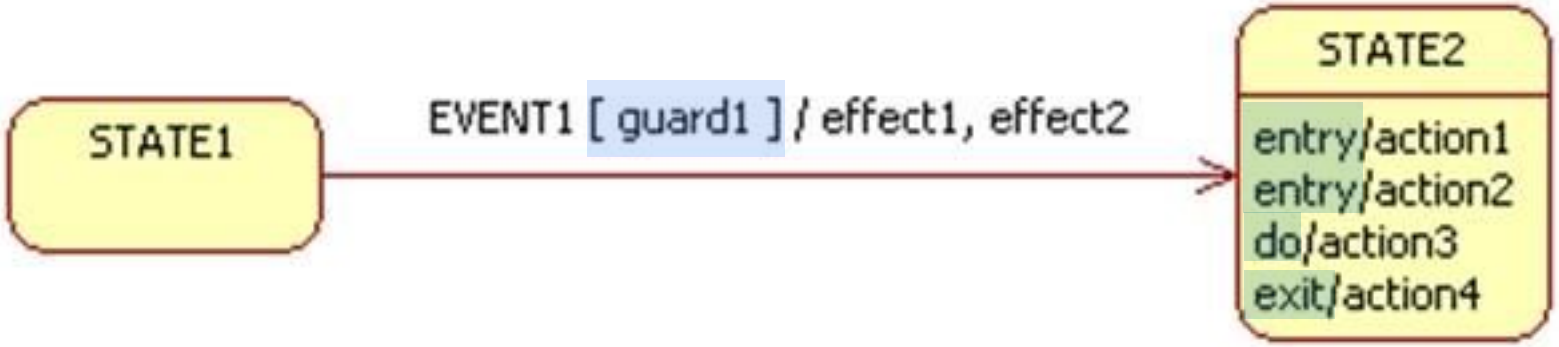
- Guardas (condiciones lógicas)
- Variables
- Estados jerárquicos (anidados)
- Estados concurrentes
- Pseudoestados
- Eventos internos

UML Statecharts

D. Harel, "Statecharts: A visual formalism for complex systems", Science of Computer Programming, vol. 8 num. 3, pp. 231–274, junio de 1987

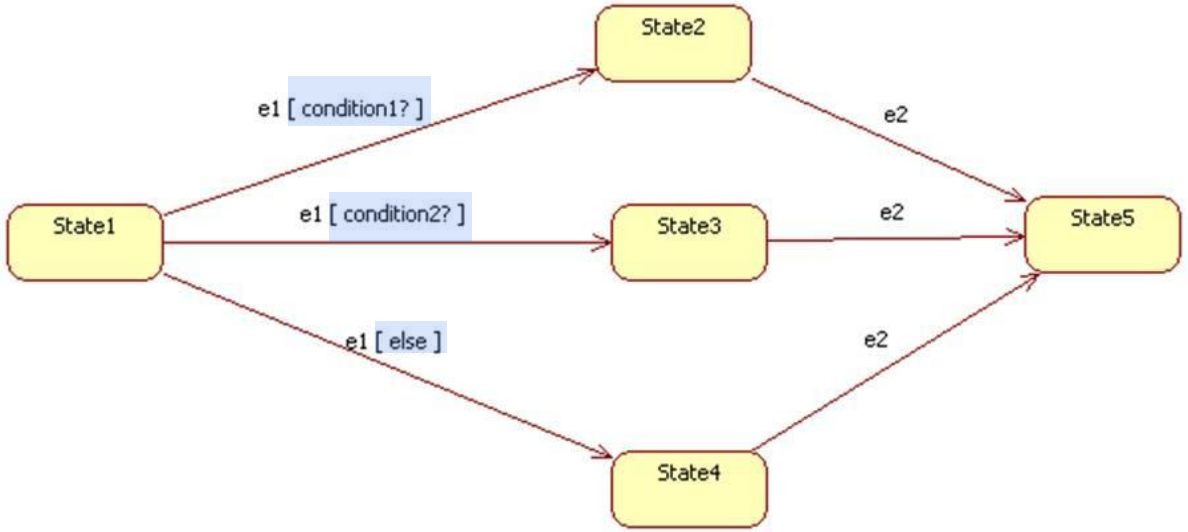
Extensiones de las FSM

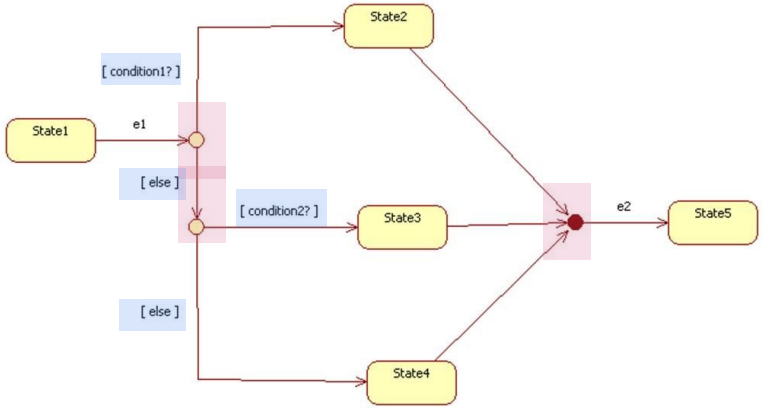
- Guardas (condiciones lógicas)
- Variables
- Estados jerárquicos (anidados)
- Estados concurrentes
- Pseudoestados
- Eventos internos



Extensiones de las FSM

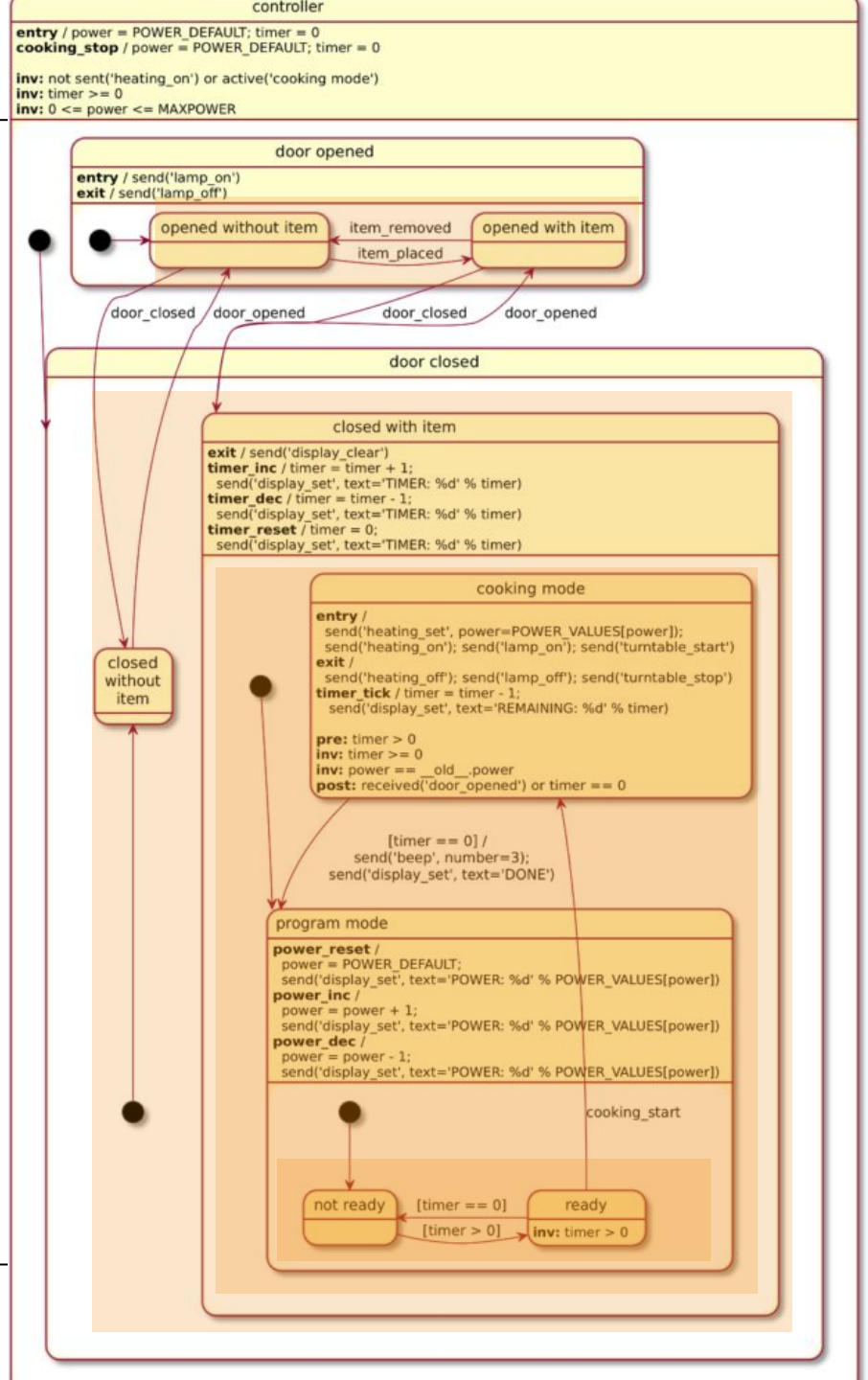
- Guardas (condiciones lógicas)
- Variables
- Estados jerárquicos (anidados)
- Estados concurrentes
- Pseudoestados
- Eventos internos



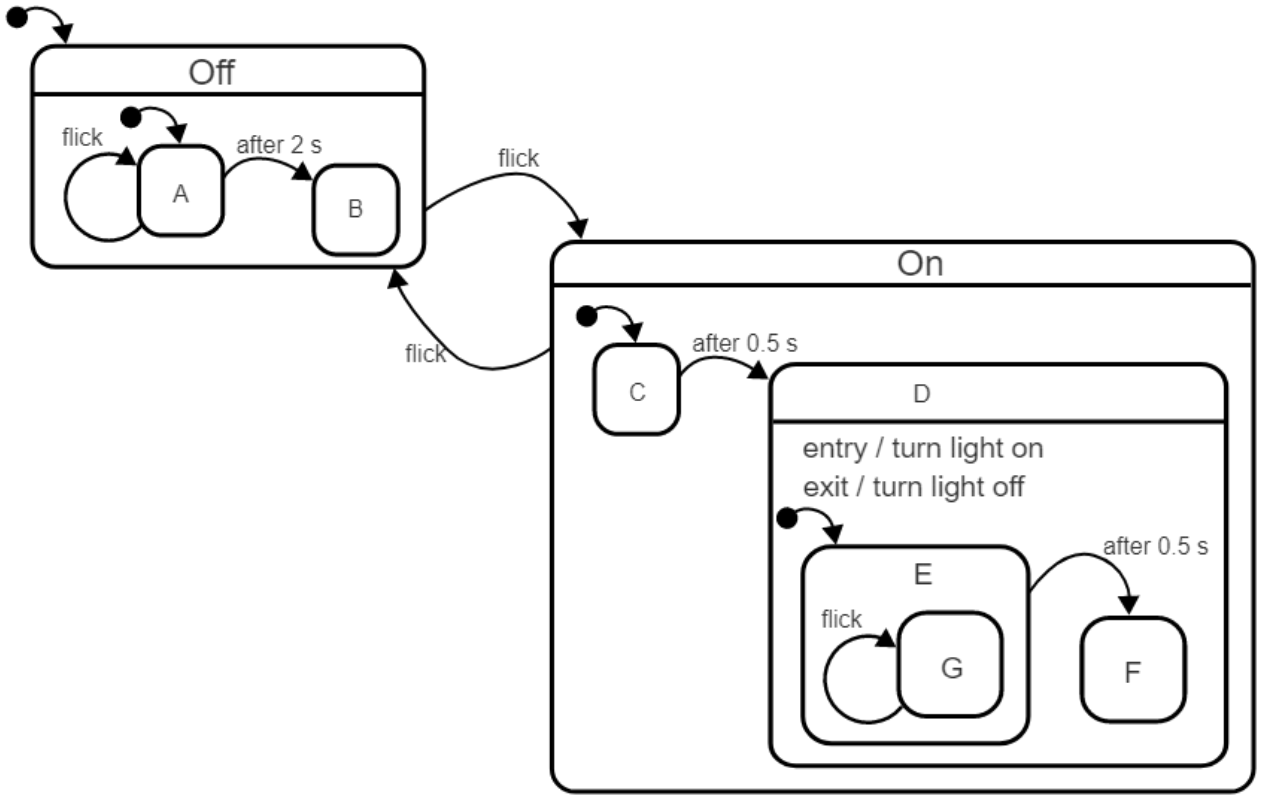


Extensiones de las FSM

- Guardas (condiciones lógicas)
- Variables
- Estados jerárquicos (anidados)
- Estados concurrentes
- Pseudoestados
- Eventos internos

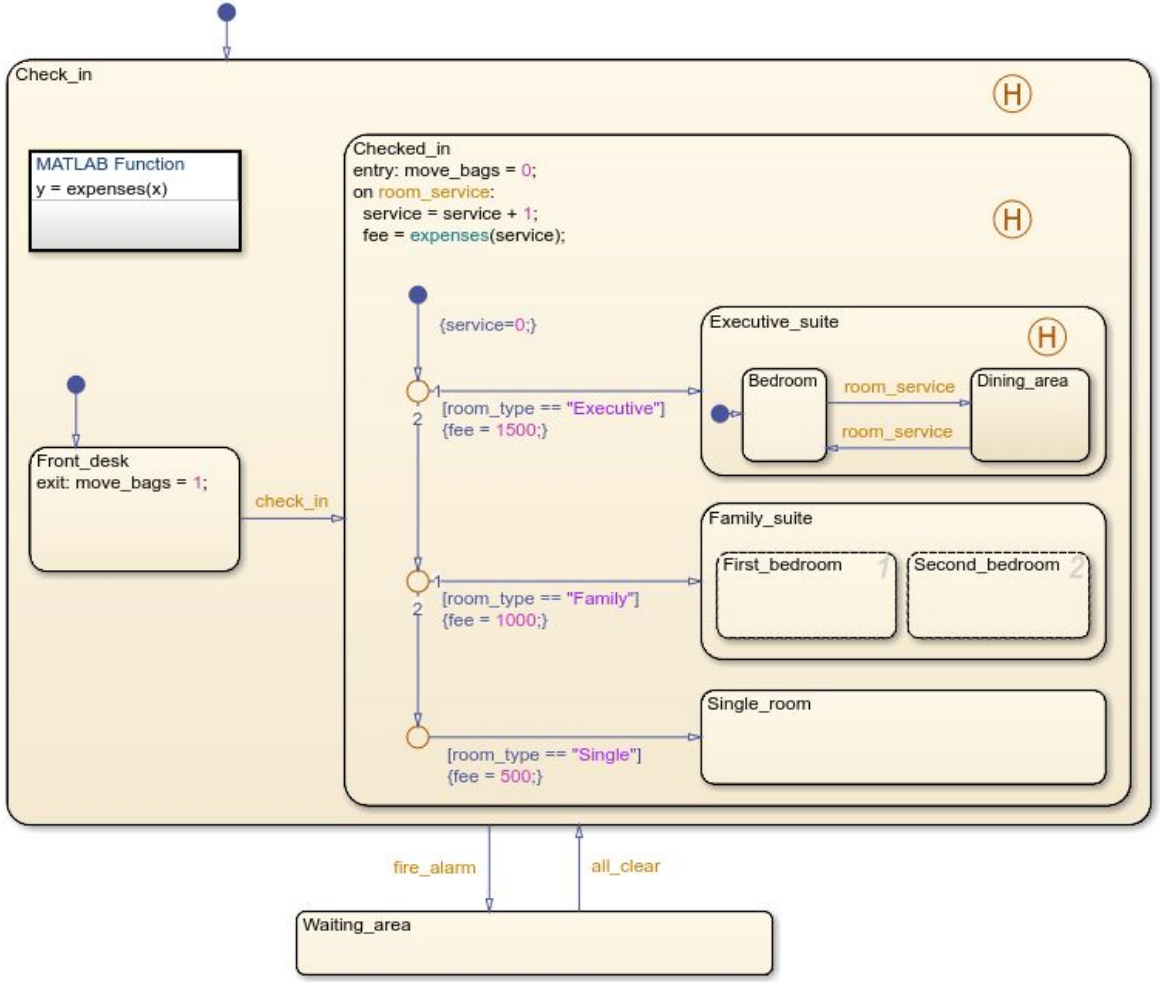


Extensiones de las FSM



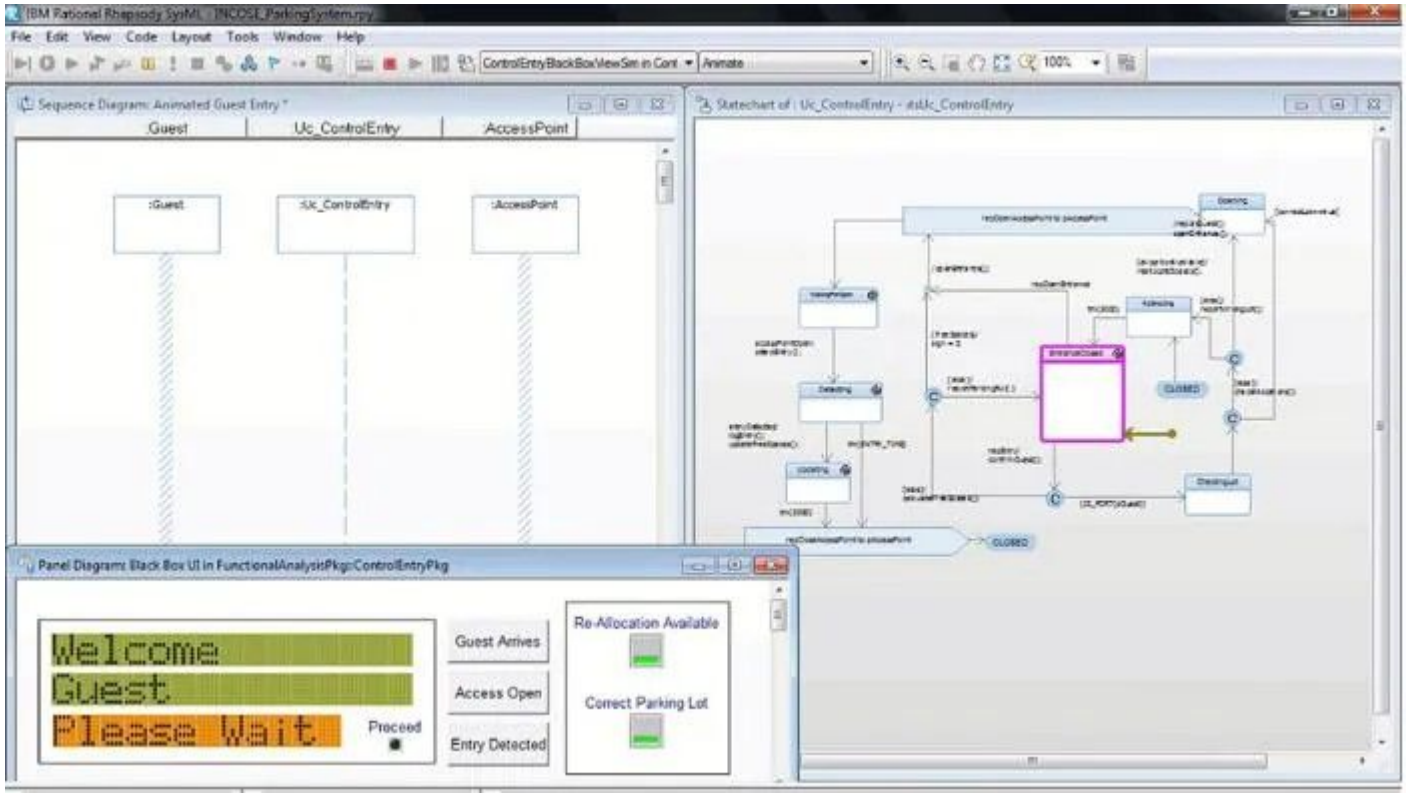
Extensiones de las FSM

Stateflow de Simulink



Extensiones de las FSM

QM Modeler de Quantum Leaps

Extensiones de las FSM

Rhapsody de IBM